

פרויקט הגמר

ניקוד: 100 מועד הגשה: סוף שיעור 12 משקל בציר הסופי: חמישים אחוז מהציון בקורס

מטרה

לבנות יישום אינטרנט אחד לניהול אוסף — שלושה עמודים, ארכיטקטורת `render ו state`, נתונים שנשמרים בדפדפן, ו-API ציבורי אחד עם כל ארבעת מצבי הממשק — ולהבין כל שורה בו.

זה חמישים אחוז מהציון בקורס, והוא נבנה **תוך כדי הקורס** ולא אחריו. כל מטלת כיתה משבוע 7 והלאה מקדמת אותו. אם עשית את המטלות, בשבוע 12 הפרויקט כמעט בנוי.

והדבר האחד שכדאי להפנים לפני שקוראים הלאה: **בשבוע 13 תשב במעבדה, 90 דקות, בלי אינטרנט, בלי AI וכלי טלפון, ותנתח את הקוד שהגשת.** שלוש מטלות ושאלת סיום: לתקן באג שהושתל בו (28 דקות · 32 נקודות), להוסיף פיצ'ר קטן (32 דקות · 33 נקודות), ולעשות `refactor` (26 דקות · 31 נקודות) — ושאלת סיום קצרה (4 דקות · 4 נקודות). זה מה שהופך את הדרישה "להבין כל שורה" ממשפט במסמך לדבר שנמדד.

מילים שעוד לא למדת – ומתי תלמד אותן

בשיעור 7 תקרא ותכתוב מילים מהעולם הזה, לפני שהקורס לימד אותן. **אל תנסה לשלוט בהן עכשיו:** בתוכנית של שיעור 7 מספיק להבין בערך על מה מדובר (בשיעור 7 קוראים מהבריוף הזה עד "מפת אבני הדרך", ומדלגים על רשימת הדרישות), והשבוע שבו כל מילה נלמדת באמת כתוב לידה. מה שנכתב כאן ידוע לך מ-C# או מ-Python, רק בלי הצורה שלו בדפדפן.

המילה	מה זה, במונחים שאתה מכיר	נלמד
endpoint ו API	שרת ציבורי שעונה על כתובות. ה-endpoint הוא כתובת אחת, כמו מתודה אחת; ה-API הוא כולן יחד	שבוע 11
JSON	הפורמט שבו השרת עונה: אובייקט או מערך כטקסט, כמו <code>dict</code> ורשימה ב-Python	שבוע 11
fetch	קריאת HTTP מהקוד, כמו <code>HttpClient</code> ב-C# או <code>requests</code> ב-Python. מחזירה <code>Promise</code>	שבוע 11
CORS	חוק של הדפדפן: עמוד מקבל תשובה משרת אחר רק אם השרת הסכים. אפשר לוודא רק מעמוד עם <code>fetch</code> , לא מלשונית	שבוע 11
localStorage	מילון מפתח-ערך שנשאר בדפדפן אחרי רענון, כמו קובץ קטן; מחרוזות בלבד	שבוע 11
render ו state	<code>state</code> הוא אובייקט אחד שמחזיק את כל מצב היישום; <code>render(state)</code> פונקציה שמציירת ממנו את המסך	שבוע 10
ישות ו- number null	הישות היא הסוג של פריט באוסף, כמו <code>class</code> עם שדות. <code>number null</code> הוא שדה שיכול להיות ריק, כמו <code>int?</code> ב-C#	הרעיון בשבוע 7, הטיפוסים בשבוע 12

המילה	מה זה, במונחים שאתה מכיר	נלמד
ערכי status	שדה אחד שמחזיק את מצב האזור, כמו enum : ארבעה מצבים — ריק · טוען · שגיאה · הצלחה; שמות הערכים בקוד נקבעים בתוכנית של שבוע 7, ובשבוע 11 בונים שדה כזה — שם הערכים אחרים (יש גם מצב לפני הבקשה הראשונה, והריק נגזר), וזה בסדר: השמות שלך	תכתוב בשבוע 7, תבנה בשבוע 11
האצלת אירועים	מאזין אחד על המכל במקום מאזין על כל פריט; האירוע "עולה" ומגלים מי נלחץ	שבוע 9
tsc ו-TypeScript	JavaScript עם טיפוסים; tsc הוא המהדר שבודק אותם ומוציא JavaScript רגיל	שבוע 12
נתון נגזר	ערך שמחשבים מה- state בכל ציור, כמו מאפיין מחושב ב-C#: הרשימה המסוננת, המונים	שבוע 10
JavaScript	השפה עצמה: משתנים, פונקציות, מערכים	שבוע 8

מה בונים

יישום לניהול אוסף. אתה בוחר את הנושא בשיעור 7 מתוך `project-topics-he.md` — 22 נושאים מאושרים — או מציע נושא משלך ומאשר אותו באותו שיעור, לא אחריו.

כל הנושאים חולקים אותה צורה, וזו לא נוחות שלי: היא מה שמאפשר למבחן בשבוע 13 לתת לכולם את אותן שלוש מטלות למרות שכל אחד בנה יישום אחר.

שלושה עמודים	רשימה · פריט בודד · עמוד שלישי שכל נתוניו נגזרים
ישות אחת	האוסף עשוי מסוג אחד של פריט
ארכיטקטורה	<code>state > render(state) > handlers</code> — מטפלים שמשתנים state
התמדה	<code>localStorage</code> , עם טעינה מגננת
רשת	API ציבורי אחד, בלי מפתח, עם ארבעת המצבים
טיפוסים	המודול <code>ts/types.ts</code> לפחות, שמתקמפל ב- <code>tsc</code>
עיצוב	Tailwind מהעותק שבמאגר. בלי CDN ובלי שלב בנייה

הדרישות

כל דרישה כאן כתובה כקריטריון שאפשר לבדוק, לא כשאיפה. "עיצוב נעים" אינו דרישה; "כל טקסט עובר 4.5:1" הוא דרישה. אם אתה לא יודע איך לבדוק אם עמדת בדרישה, שאל אותי בשיעור — כנראה גם אני לא כתבתי אותה טוב.

א. ארכיטקטורה — הדרישה שהמבחן נשען עליה

- יש אובייקט `state` אחד, והוא היחיד שמחזיק את מצב היישום.
- יש פונקציה `render(state)` שמציירת את המסך מהאובייקט הזה, והיא נקראת בכל שינוי.
- מטפלי אירועים משנים `state` וקוראים ל-`render`. אין מטפל שמצייר את התוצאה בעצמו.

4. הקוד מחולק לשכבות, קובץ לכל אחת: `state` · `render` · `events` · `storage` · `api` - `js/state.js` מייצא שלושה שמות בדיוק כפי שהם כתובים כאן: `subscribe` · `setState` · `getState` — הם מגיעים כתובים בתיקית הפתיחה, ואין צורך לכתוב אותם. רק לא לשנות את שמם.
5. כל מה שאפשר לגזור — נגזר, לא נשמר. הרשימה המסוננת, המונים, הסיכומים, רשימת הקטגוריות לסינון: כולם פונקציות של `state`, לא שדות בו.
6. מאזינים נרשמים על מכל אחד (האצלה), לא על פריטים. פריט נמחק בכל רינדור, ומאזין שנרשם עליו מת איתו.
7. `render(state)` לא קורא את ה-DOM כדי להחליט מה לצייר. הוא קורא רק את `state`.

למה דווקא שלושת השמות האלה, כשכל שאר השמות בפרויקט שלך

כי הם הדלת היחידה שדרכה מישוהו אחר יכול להפעיל את היישום שלך בלי לקרוא אותו. הבודק האוטומטי מפעיל את הפרויקט דרכם — 67 מתוך 100 הנקודות נמדדות ככה — ובשבוע 13 התקלה מושלתל דרכם. כל שם אחר בפרויקט חופשי לחלוטין: המזהים, המחלקות, שדות הישות, שמות הגזירות. שלושת אלה לא. סטודנט שקרא ל-`setState` בשם `update` בנה יישום תקין לגמרי ואי אפשר לבחון אותו, וזה מתגלה בחזרה הגנרלית בשבוע 12 ולא לפני.

שים לב

דרישה 5 היא זו שנופלים בה, והיא נופלת בשקט. שדה `filteredItems` לצד `items` עובד מושלם בכל הדגמה — ונשבר ברגע שמשהו משנה את `items` בלי לזכור לחשב אותו מחדש. מה שיישכח יהיה בדיוק מה שמוצג על המסך.

ב. שלושת העמודים

- עמוד רשימה (`index.html`): האוסף כולו, עם חיפוש טקסט, סינון אחד לפחות, ומיון בשני מפתחות לפחות. שלושתם נגזרים מ-`state`. הבודק האוטומטי מפעיל סינון ומיון דרך `<select>`; סינון או מיון בכפתורי רדיו, בתיבות סימון, בכפתורים (גם `button` פשוט עם `data-action`) או בקישורים נבדק ביד, ואינו מפסיד.
- עמוד פריט (`item.html?id=` או שם דומה): פריט אחד לפי מזהה מה-URL, עם עריכה. מזהה שאינו קיים מציג מצב ריק עם דרך חזרה — לא שגיאה, ולא עמוד לבן.
- עמוד שלישי שכל נתוניו נגזרים מהאוסף ואין לו נתונים משלו: סיכום, פילוח, לוח מספרים. עמוד "אודות" אינו עומד בדרישה.
- ניווט מלא בין שלושת העמודים, ומכל עמוד יש דרך חזרה. בעמוד שמופיע בתפריט הניווט, הקישור אליו נושא `aria-current="page"` כשאתה נמצא בו. עמוד הפריט מגיעים אליו מהרשימה ולא מהתפריט, ולכן הוא לא חייב להופיע בתפריט — אבל הוא חייב תפריט שמוביל חזרה.

ג. CRUD והתמדה

- הוספה · עריכה של שדה אחד לפחות · מחיקה. שלושתם משנים `state` ונשמרים.
- הכול שורד רענון של הדפדפן.
- המפתח ב-`localStorage` מרוחב שם וממוספר גרסה: `<app>:v1`. מפתח כמו `items` מתנגש עם כל פרויקט אחר שהוגש מאותו `localhost`. ויש נתיב קריאה אחד בלבד: ב-`js/storage.js` יש קריאה אחת ל-`getItem` שקוראת את האוסף. אם אתה שומר עוד משהו — העדפה, מיון אחרון, ערכת נושא — הוא נקרא בפונקציה משלו ולא באותה שורה, ועדיף שלא ב-`storage.js` בכלל. שתי קריאות שמחזירות "את הנתונים" הן שני מקורות אמת בדיוק כמו שדה נגזר שנשמר, וההבדל הוא רק שהן על הדיסק. והחזרה של הערך: מה שנשמר תחת המפתח הוא המערך עצמו (`JSON.stringify(items)`), לא אובייקט שעוטר אותו; ומפתח שאינו קיים הוא אוסף ריק, לא שגיאה. הבודק ומבחן שבוע 13 נשענים על שני אלה.

4. **טעינה מגננת.** שלושת המקרים האלה לא מפילים את היישום ולא מוחקים נתונים:

- הערך אינו JSON תקין
- הערך הוא JSON תקין אבל אינו מערך
- הערך הוא מערך שיש בו שורה אחת בפורמט לא נכון — היא מושמטת, השאר נטען

5. **כתיבה שנכשלה נאמרת למשתמש.** `localStorage` מוגבל בנפח, ובמצב פרטי הוא יכול לזרוק. יישום שמניח שהשמירה הצליחה מציג מדף שלא יהיה שם מחר.

ד. הרשת וארבעת המצבים

1. **API ציבורי אחד**, בלי מפתח, שמחזיר JSON ומרשה CORS. הרשימה שנבדקה נמצאת ב- `project-topics-he.md`.

2. לכל אזור שמציג נתונים מהרשת יש **ארבעה מצבים**, ולכל אחד מהם משהו להראות:

מצב	הדרישה
ריק	טקסט שאומר מה לעשות. "עוד לא חיפשת" ו"החיפוש לא מצא" הם שני מצבים ריקים שונים
נטען	סימן חזותי שמשהו קורה, עם פסק זמן . ספינר בלי פסק זמן מסתובב עד רענון
שגיאה	מה קרה, במשפט שאפשר לפעול לפיו, ודרך לצאת ממנו (נסה שוב, או מסלול חלופי)
הצלחה	הנתונים

3. **res.ok נבדק תמיד.** `fetch` אינו נדחה על 404 או 500 — הוא מצליח, ו- `res.ok` הוא `false`. 500 שמטופל כהצלחה מייצר עמוד שמרנדר מסמך שגיאה.

4. **AbortController עם פסק זמן.** רשת שמקבלת חיבור ולא עונה תולה את היישום לנצח. **הגבול:** בין 3 ל-15 שניות מהבקשה, כולל ניסיון חוזר אם יש (בקורס השתמשנו ב-8). פסק זמן ארוך יותר נראה לבודק כמו ספינר בלי פסק זמן.

איך הבודק מגיע לרשת

ארבע השורות של המצבים (דרישה 18) מפעילות את ה-API שלך בהקלדה בשדה טקסט ו-Enter — החיפוש, כמו במחזורי שבוע 11. אם ה-API שלך מופעל אחרת (למשל נושא 8, תחזית לפי מיקום של פעילות), הבודק לא ימצא בקשה, והשורות האלה יסומנו לבדיקה ביד — הן לא נספרות כאפס. **והתשובה שהבודק מחזיר לבקשה שלך היא הקובץ `data/catalogue.json` שבפרויקט שלך** — הקטלוג המקומי של דרישה 21, שמגיע בתיקיית הפתיחה. השאר אותו בשם ובמקום האלה, עם שורות אמיתיות בצורה שה-API שלך מחזיר. בלעדיו אין לבודק תשובה להשיב בה, ושורת מצב ההצלחה נופלת.

21. **אין קריאה לרשת שמפילה את העמוד כשאין רשת.** בשבוע 13 אין אינטרנט. יישום שמראה עמוד לבן בלי חיבור אינו עומד בדרישה — קובץ JSON מקומי כמסלול חלופי הוא פתרון טוב, והוא פיצ'ר ולא תוכנית מגירה.

ה. TypeScript

1. **המודול `ts/types.ts`** (הוא מגיע בתיקיית הפתיחה; מודולים נוספים ב- `ts/` רשות), עם `tsconfig.json`, שמתקמפל ב- `npx tsc` בלי שגיאות. הבודק קורא אותו בשם הזה.

2. הוא מכיל `interface` או `type` לישות שלך, **ולפחות שומר אחד בזמן ריצה**
(`function isThing(value: unknown): value is Thing`), או כפונקציית חץ
(`export const isThing = (value: unknown): value is Thing =>`

3. הדפדפן טוען את **הפלט** (`js/`), לא את המקור — ובפרט `js/types.js`, הפלט של `ts/types.ts`. הפלט מקומם בגיט.

דרישה 23 היא הסיבה שהמודול הזה שווה משהו. טיפוסים נעלמים בקומפילציה – הם לא בודקים כלום בזמן ריצה, ולכן הם לא יכולים להגן עליך ממה שיש ב- `localStorage` של מישוהו. השומר הוא הקוד היחיד בקובץ שבאמת רץ. פתח את הפלט והסתכל.

ו. עיצוב, נגישות ורספונסיביות

1. **Tailwind מהעותק המקומי** (`vendor/tailwind.js`). לא CDN – בשבוע 13 אין רשת.
2. שכבת טוקנים ב- `@theme`, וכל צבע נגזר ממנה. **אין צבע מוקלד ידנית בסימון**.
3. הכול על **הסולם**: מרווחים, גדלים וטיפוגרפיה מהסקאלה, בלי ערכים שרירותיים – אלא אם יש סיבה, ואז היא כתובה בהערה.
4. **1280 • 768 • 375**, בלי גלילה אופקית ובלי פריסה שבורה באף אחד מהם.
5. **מקלדת בלבד**: אפשר להשלים את המסלול המרכזי כולו בלי עכבר; כל פקד מקבל פוקוס; לכל פקד ממוקד יש **טבעת נראית**; סדר הפוקוס הגיוני.
6. **outline: none** בלי חלופה נראית פוסל את הסעיף הזה. אין יוצא מן הכלל.
7. **WCAG AA**: כל טקסט 4.5:1, טקסט גדול 3:1, וגבול של פקד שצריך לזהות אותו 3:1.
8. **alt** שאומר משהו – או `alt=""` לתמונה דקורטיבית. `alt` חסר גורם לקורא מסך להקריא את שם הקובץ.
9. סדר כותרות תקין, `h1` אחד לעמוד, בלי דילוג על רמה.
10. צבע לבד לא נושא משמעות. סטטוס שמסומן רק בנקודה אדומה או ירוקה אינו נגיש.
11. **prefers-reduced-motion** מכובד בכל מקום שמשוהו זז.
12. `axe` על שלושת העמודים: **אפס הפרות** ברמות `wcag2a` ו- `wcag2aa`.

ז. איכות קוד והיגיינת ריפו

1. אפס שגיאות ואפס אזהרות בקונסול בטעינה ובמסלול המרכזי.
2. HTML תקין בשלושת העמודים.
3. **textContent** ולא **innerHTML** לכל דבר שמגיע ממשתמש או מהרשת – או escaping מפורש. זה לא סעיף תיאורטי: שדה `summary` ב-API אחד לפחות מהמאגר מחזיר HTML.
4. קומיטים קטנים עם הודעות שאומרות משהו, לאורך כל התקופה. ריפו שכל הפרויקט בו בקומיט אחד מסוף שבוע 12 מאבד את הסעיף הזה – וגם לא ישכנע אף אחד שהוא שלך.
5. `README.md` בעברית: מה היישום עושה, איך מריצים אותו, ה-API שבו השתמשת, ומה לא בנוי.

מדיניות AI לשבוע הזה

משבוע 8 והלאה: AI הוא שותף מלא ליישום. מותר ומצופה שתשתמש בו. מה שנמדד הוא לא אם השתמשת, אלא **אם אתה יודע מה יש בקוד שלך**. ביקורת על פלט של AI היא מקצוע נלמד, והיא שורה בטבלת הניקוד — לא הסתייגות בסוף. בשכבת השותף, מסלול הצ'אט: **בכל מקום שכתוב "Claude Code", אפשר להדביק את המפרט ל-Claude בדפדפן, להעתיק את הקבצים ידנית, ולעשות בדיוק אותה ביקורת.** התוצר זהה, והנקודות זהות. אין לך מנוי? הכול עובד גם בצ'אט החינמי, ואין בכך שום חיסרון בקורס.

ארבעה מסמכים מוגשים עם הקוד. התבניות והדוגמאות המלאות ב- `project/templates/`.

המסמך	מתי	מה יש בו
PROJECT_PLAN.md	שיעור 7	הנושא, האוסף, השדות, ה-API, ארבעת המצבים, צורת ה-state, לוח זמנים. מתעדכן לאורך הדרך
specs/<feature>.md	לכל פיצ'ר	מפרט לפני קוד, ובסופו ביקורת על מה שחזר
PROMPTS.md	לאורך הדרך	יומן: מה ביקשת, מה חזר, מה היה שגוי , ומה למדת על הפרומפט
AI_USAGE.md	סוף שבוע 12	לפי קובץ: כמה בעזרת AI, והאם אתה יכול להסביר כל שורה

מה מאמת את ההצהרה: מבחן הניתוח. אין דרך אחרת ואין צורך באחרת. התקדים הוא שלי, מהבריי של הפרויקט הקבוצתי הקודם: "סטודנט שלא מסוגל להסביר קטע קוד שלו עלול להפסיד את כל הניקוד האישי של חלק הקוד." מבחן הניתוח הוא אותו כלל, נאכף על כולם באותה מידה, ומאומן שלוש פעמים לפני כן — אתגר הדיבאג בשבוע 9, מיני-ניתוח בשבוע 10, וניתוח מלא בשבוע 12. `AI_USAGE.md` שימושי לך בדבר אחד: הוא מכריח אותך, שבוע לפני המבחן, לעבור על הקוד ולסמן איפה יש חורים — כשעוד אפשר לסגור אותם. **סעיף "מה אני מבין רק חלקית" שנשאיר ריק בשקר הוא הדבר היחיד במסמך הזה שעלול לעלות לך ביושרה.**

מפת אבני הדרך

בסוף כל שבוע יש מה שצריך להיות בריפו. העמודה השלישית היא האבחון: אם היא נכונה לגביר, אתה מאחר — לא "כמעט".

שבוע	מה צריך להיות בריפו בסוף השבוע	אתה מאחר אם
7	הנושא נסגר. <code>PROJECT_PLAN.md</code> מלא ומקומם, כולל סעיף 11 — הביקורת על ה-AI	אין לך נושא, או שהתוכנית היא כותרות בלי שדות
8	שלושת העמודים קיימים כ-Tailwind+HTML, עם נתונים מומצאים בסימון	יש עמוד אחד, או שאין תפריט ביניהם
9	הממשק מגיב: האצלה על המכלים, <code>preventDefault</code> על הטופס	הכפתורים לא עושים כלום, או שכל מאזין נרשם על פריט

שבוע	מה צריך להיות בריפוי בסוף השבוע	אתה מאחר אם
10	render.js + state.js . האוסף מוצג מ- state , לא מסימון קבוע	הרשימה עדיין כתובה ב-HTML
11	CRUD מלא + localStorage + טעינה מגננת · ה-API מחובר, ארבעת המצבים עובדים ואפשר להראות כל אחד מהם	הנתונים נעלמים ברענון, או ש-JSON פגום מפיל את העמוד · יש ספינר בלבד, או שאין מצב שגיאה
12	מודול TS מתקמפל · מקלדת · AA · ארבעת המסמכים · README	ההגשה

ההגשה: ב-Moodle, בסוף שיעור 12 — כתובת הריפוי וה-SHA של הקומיט. ה-SHA הוא מה שמקפיא את ההגשה: דחיפות אחריו לא משנות את מה שנבדק, ואין על זה ויכוח. זה גם מה שמאפשר לי לטעון את הקוד שלך למכונות המעבדה לפני שבוע 13.

מה לא נדרש, ולא ייתן נקודות

יש כאן דברים שסטודנטים משקיעים בהם שבוע ומקבלים עליהם אפס. חבל.

- **מסגרת, באנדלר, או ספרייה** — אסורים (tsc הוא היוצא מן הכלל היחיד). כולל ספריות גרפים, ספריות תאריכים וספריות אנימציה. גרף עמודות הוא טבלת HTML עם רוחב באחוזים, והוא נגיש יותר מ-canvas.
- **התחברות ומשתמשים** — אין שרת, אין מה לאבטח, ואין מה להדגים.
- **שרת, בסיס נתונים או API משלך** — זה קורס צד לקוח.
- **לוח שנה שנבנה מאפס** — `<input type="date">` נגיש, מתורגם ועובד במקלדת.
- **Drag & Drop** — לא נלמד, ולא נגיש בלי עבודה נוספת. כפתורי "העלה/הורד" קודם; גרירה רק אחריהם, ורק כתוספת.
- **יותר משלושה עמודים** — עמוד רביעי לא מוסיף ניקוד. הוא מוסיף שלושה מקומות שהתפריט יכול להישבר בהם.

נקודות מעבר, ומה נבדק אוטומטית

הציון בנוי מ-100 נקודות. הפירוט המלא בטבלת הניקוד שיחולק בשיעור 7 יחד עם המסמך הזה; מה שכדאי לדעת עכשיו:

- **חלק מהדרישות נבדקות אוטומטית** — הארכיטקטורה, ארבעת המצבים, ההתמדה, הרספונסיביות, המקלדת, הניגודיות ו-axe .
הן נמדדות על העמוד המרונדר, לא על מחרוזות בקוד: אם הגעת לתוצאה בדרך אחרת, עברת. **גם איכות הקוד ברובה אוטומטית:** 10 מתוך 12 הנקודות שלה (קונסולה שקטה, HTML תקין, tsc עם שומר בזמן ריצה, בלי מסגרות, escaping), ו-2 ידניות (קוד קריא).
- **כל סימון CODE HERE של הליבה חייב להיעלם מהקבצים** js/events.js · js/render.js · js/state.js · ts/types.ts · js/api.js · js/storage.js . סימון אחד שנשאר מאפס את כל השורות האוטומטיות — הבודק לא יכול להבחין בינו לבין תיקיית פתיחה שלא נגעו בה. סימון שבשורה שלו כתוב גם {optional} , **הרחבה** או **אתגר** מותר להשאיר אם דילגת על החלק הזה. הבדיקה העצמית (node . checks/run-checks.mjs) אדומה כל עוד נשאר סימון.
- **חלק ידני** — העיצוב, קריאות הקוד, ארבעת המסמכים, והיגינת הריפוי. בעיצוב יש גם שורה על **פעולות הרסניות**: מחיקה מקבלת ביטול (undo) או אישור — שניהם מקבלים ניקוד מלא, ומחיקה בלי אף אחד מהם מאבדת את השורה.
- **מעל המפרט — עד 8 נקודות, ידני**, על דברים שהמפרט לא ביקש ומראים שליטה: מצב כהה שעובר AA, ביטול במקום דיאלוג אישור, שכבת טוקנים אחת לשלושת העמודים, מסלול אמיתי למצב בלי רשת, בדיקה אוטומטית שכתבת בעצמך, ונגישות מעל הנדרש (ניהול פוקוס אחרי מחיקה, אזור חי שמכריז נכון). לא עוד פיצ'רים — שליטה.

• CI ירוק הוא הרצפה, לא הציון. הוא מוכיח שהדבר רץ. הנקודות מגיעות ממה שמכונה לא יכולה לשפוט.

הגשה

Moodle • סוף שיעור 12 • כתובת הריפוי + SHA של הקומיט. הפרויקט יושב בתיקיית `project/` בריפוי הסמסטריאלי שלך – אותו ריפוי פרטי אחד מכל הסמסטר, שאני שותף בו משבוע 1.

הגשה מאוחרת: ההגשה נסגרת במועד שכתוב במודל. הגשה שמגיעה אחרי המועד מתקבלת ונבדקת במלואה, ומסומנת כמאוחרת – היא אינה מורידה נקודות.